Linux & Bash Basics

"Or: How to actually use your system"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

Linux & Bash Basics

"Or: How to actually use your system"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

1 The Linux Operating System

"*From:torvalds@klaava.Helsinki.FI (Linus Benedict Torvalds)*
Newsgroup: comp.os.minix
Subject: What would you like to see most in minix?
Summary: small poll for my new operating system
Message-ID: 1991Aug25, 20578.9541@klaava.Helsinki.FI
Date: 25 Aug 91 20:57:08 GMT
Organization: University of Helsinki.
Hello everybody out there using minix-

I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones. This has been brewing since april, and is starting to get ready. I'd like any feedback on things people like/dislike in minix; as my OS resembles it somewhat (same physical layout of the file-sytem due to practical reasons)among other things."

The Linux Operating System - History

- 1969: creating of the UNIX operating system
  - well loved, but code was owned by AT&T
- 1983: GNU project was created, as free and open replacement for UNIX
  - creating tools that we know today (`tar`, `grep`, `emacs`, ..)
  - was missing an OS kernel (glue between hardware and software)
- 1991: Linux was used as the OS kernel, resulting in the operating system GNU/Linux

The Linux Operating System - Features

- free
- open source
- allowed developers to build custom OS
  - a so-called distribution
    - Linux kernel
    - Packages (GNU)
    - package manager
    - Desktop UI
  - customized to serve different purposes
    - enterprise vs. home
    - server vs. desktop vs. mobile vs. embedded devices
  - well known distros:
    - Debian (children: Ubuntu, SteamOS, ...) → apt
    - Redhat (children: RHEL, CentOS, Fedora, ...) → rpm / yum
    - Arch (children: Manjaro, ...) → pacman
    - independent distros (Alpine Linux, Fire OS, Android)

Linux & Bash Basics

"Or: How to actually use your system"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

2 Linux File Structure - /

- Linux file structure can be thought of a hierarchical tree
- origin directory is called the “*root*” directory and represented by /
- root directory contains file and further directories, which can contain files and directories themselves … and so on and so forth
- regarding the representation, we can represent the location of a file like so

```
/firstdirectory/seconddirectory/thirddirectory/example.txt
```

2 Linux File Structure - Overview

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

2 Linux File Structure - /bin

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/bin

- contains binaries of the system

2 Linux File Structure - /boot

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/boot

- contains files required for booting the system

2 Linux File Structure - /dev

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/dev

- contains device files
- devices are represented as files
- examples
  - webcam

2 Linux File Structure - `/etc`

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

`/etc`

- originally named after “*et cetera*”; place to dump things you were not sure where to place
- more fitting now: “*everything to configure*”
- contains many system-wide configuration files, for example
  - `/etc/hostname`
  - `/etc/hosts`
  - `/etc/passwd`
  - `/etc/resolv.conf`
  - `/etc/shadow`

2 Linux File Structure - /home

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/home

- contains personal directories of users
- general structure of personal directories:

```
[maria@linux ~]$ tree -L 2 .
.
└── maria
    ├── Desktop
    ├── Documents
    ├── Downloads
    ├── Music
    ├── Pictures
    ├── Public
    ├── Templates
    └── Videos
```

2 Linux File Structure - `/lib`

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

`/lib`

- contains shared libraries, i.e., files that contain code used by applications
- also contains kernel modules

2 Linux File Structure - /media

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/media

- used for manually mount external removable media devices such as USB drives

2 Linux File Structure - /mnt

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/mnt

- temporary mount point for filesystems
- can be used by administrator to repair

2 Linux File Structure - /opt

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/opt

- stores additional software packages, not part of core system

2 Linux File Structure - /proc

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/proc

- virtual filesystem generated by the kernel
  - virtual directories and files are special (always size 0, even if big)
  - file names represent ID of associated process
- files represent processes

2 Linux File Structure - `/root`

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

`/root`

- personal directory of the `root` user
- only accessible by the `root` user

2 Linux File Structure - /run

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/run

- system processes use it to store temporary data

2 Linux File Structure - `/sbin`

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

`/sbin`

- like `/bin`, but contains binaries that require superuser (root) privileges

2 Linux File Structure - /srv

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/srv

- may contain directories related to servers and what they are serving
- for example:

```
[maria@linux ~]$ tree -L 2 /srv
/srv
├── http
├── ftp
└── smb
```

2 Linux File Structure - /sys

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/sys

- Provides file system-like view of information and configuration settings that the kernel provides

2 Linux File Structure - /tmp

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/tmp

- contains temporary files
- every user can write and read files in this directory

2 Linux File Structure - /usr

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/usr

- Unix System Resources
- contains
  - binaries that are not system critical
  - libraries
  - software you installed manually
  - ...

2 Linux File Structure - /var

```
[user@linux ~]$ tree -L 1 /
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

/var

- holds variable data
  - mails
  - caches
  - databases
  - logs

2 Linux File Structure - Special Directory Names

- `.` → current directory
- `..` → parent directory
- `~` → home directory of current user

- every directory contains `.` and `..`
  - what does `..` of `/` point to?

- absolute path → starting from the root directory
- relative path → starting from the current directory

Linux & Bash Basics

"Or: How to actually use your tools"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

3 Bash Basics - Shells

- shell = command-line interface that allows user to interact with the operating system
  - Sh (Bourne Shell) → /bin/sh
  - Bash (Bourne Again SHell) → /bin/bash
  - Zsh (Z Shell) → /usr/bin/zsh

- pick your poison, but know the basics in all of them
  - Vim
  - Nano
  - Emacs

3 Bash Basics - Common Environment Variables

- `HOME` → home directory of current user
- `USER` → username of currently logged-in user
- `PATH` → list of directories used for searching for executables
- `SHELL` → path to default shell
- `PWD` → present working directory
- `EDITOR` → default text editor
- `HOSTNAME` → hostname of the machine

- use `$<variable>` to reference them

3 Bash Basics - Shebang

- `#!`
- placed at the very beginning of a script file that tells the OS which interpreter to use

```
#!/bin/bash
```

```
#!/usr/bin/python3
```

```
#!/usr/bin/perl
```

3 Bash Basics - Flags/Switches of Commands

- most Linux commands accept options (also called flags or switches) that change how they work
- options usually start with a `-` (single letter) or `--` (word form)

```
$ ls -l                 # long list format
$ ls -a                 # show hidden files
$ ls -la                # combine both
```

- when it comes to documenting the usage of a command, there are multiple syntax directives that can be used

```
$ ls [-l] <directory>          # [*] indicate something optional
                               # <*> indicates something to be
                               #   replaced
$ mkdir project/{src,plan}     # {a,b} indicates a set of choices
```

information commands

```
# display documentation of utility
[user@linux ~]$ man <command>

# display help text of a utility
[user@linux ~]$ <command> --help

# display version of utility
[user@linux ~]$ <command> --version

# display command history
[user@linux ~]$ history

# determine file type
[user@linux ~]$ file <file>
```

navigation commands

```
# show current directory
[user@linux ~]$ pwd

# move to a directory
[user@linux ~]$ cd <directory>

# list files and directories
[user@linux ~]$ ls [<directory>]

# list all files and directories (including hidden) with
# detailed info
[user@linux ~]$ ls -la
```

file and directory management 1

```
# copy files or directories
[user@linux ~]$ cp <source> <destination>

# move or rename files or directories
[user@linux ~]$ mv <source> <destination>

# remove a file
[user@linux ~]$ rm <file>

# remove a directories recursively and forcefully
[user@linux ~]$ rm -rf <directory>
```

file and directory management 2

```
# create a new directory
[user@linux ~]$ mkdir <directory_name>

# remove empty directories
[user@linux ~]$ rmdir <directory_name>

# create an empty file
[user@linux ~]$ touch <file>
```

file viewing

```
# display contents of a file
[user@linux ~]$ cat <file>

# scroll through a file interactively
[user@linux ~]$ less <file>

# view file content page by page
[user@linux ~]$ more <file>

# display first few lines of a file
[user@linux ~]$ head <file>

# display last few lines of a file
[user@linux ~]$ tail <file>
```

search and locate

```
# locate a program's executable path
[user@linux ~]$ which <command>

# search inside files for matching text patterns
[user@linux ~]$ grep "search_pattern" <file>

# locate a file using a pre-built database
[user@linux ~]$ locate <file>

# locate a file in real time, recursively by ...
[user@linux ~]$ find <path> -name <filename>     # name
[user@linux ~]$ find <path> -type <filetype>     # type
[user@linux ~]$ find <path> -size <filesize>     # size
```

3 Bash Basics - Pipe

- |
- used to chain commands

```
$ cat data.txt | grep "keyword" | sort | uniq
```

3 Bash Basics - Asterisk Character

- *
- used to match any number of characters in filenames
- Bash uses this when expanding file names (called *globbing*)

```
$ ls *.txt      # shows all files ending with .txt
$ ls test*      # shows all files starting with "test"
$ ls *file*     # shows all files that have "file" anywhere in
                #     the name
```

3 Bash Basics - Variable Expansion Character

- $
- used to get the value of a variable

```
$ user="alex"
$ echo $user    # prints alex
```

3 Bash Basics - Escape Character

- \
- tells Bash to treat the next character literally, even if it is special

```
$ echo "He said \"Hi\""  # prints: He said "Hi"
$ echo \$HOME            # prints: $HOME
```

3 Bash Basics - Strings with ‘ and “

- both ‘ and “ are used to declare strings in Bash
- the type of quote controls how special characters are handled inside strings
  - single quotes: everything inside is taken literally (no variable expansion or special meaning)
  - double quotes: variables and special characters like $ are expanded.

```
$ name="Sam"
$ echo 'Hello $name'       # prints: Hello $name
$ echo "Hello $name"       # prints: Hello Sam

$ echo "He said \"Hi\""    # prints: He said "Hi"
```

3 Bash Basics - Bash related Files

- `~/.bashrc`
  - runs every time you open a terminal
  - used to set up your Bash environment (like environmental variables, aliases, ...)
- `~/.bash_profile`
  - runs once when you log in
  - used for session-wide settings
  - will often run `.bashrc` to import usual settings
- `~/.bash_history`:
  - keeps a record of commands you typed
    - you can view it with `history`
  - allows to reuse old commands
    - `!!` → execute the last command

Linux & Bash Basics

"Or: How to actually use your system"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

4 Stream Control

- in Linux, there are three standard data streams
  - STDIN (standard input, file descriptor → `0`)
    - input stream
  - STDOUT (standard output, file descriptor → `1`)
    - output stream, by default writes to terminal
  - STDERR (standard error, file descriptor → `2`)
    - stream for error messages, by default also writes to terminal

input/output redirection

```
# redirect STDOUT to a file, overwrite
[user@linux ~]$ echo "hello" > output.txt

# redirect STDOUT to a file, append
[user@linux ~]$ echo "hello" >> output.txt

# redirect STDIN from a file
[user@linux ~]$ wc -l < input.txt
```

filters

```
# search for patterns
[user@linux ~]$ cat file.txt | grep "keywords"

# cut each line into sections, and return the second section
[user@linux ~]$ cat file.txt | cut -d ',' -f 2

# sort lines of text
[user@linux ~]$ cat file.txt | sort

# return all unique lines in a file
[user@linux ~]$ cat file.txt | sort | uniq

# translate characters
[user@linux ~]$ cat file.txt | tr 'a-z' 'A-Z'
```

Linux & Bash Basics

"Or: How to actually use your system"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

5 Users, Groups & Permissions - Users

- user: represents an entity that can log in or execute commands
- each user ...
  - is defined by their UID (user id)
  - is assigned a default shell
  - has a home directory of some type (optional)
- `root` is the special superuser
  - uid of 0
  - unrestricted access to all resources and commands
- generally, users can be divided into two types
  - system users: user accounts used for system or services (uid 0 - 999 usually)
  - regular users: humans (uid beginning from 1000)

5 Users, Groups & Permissions - Groups

- group: represents a collection of users
- main purpose is for managing permissions
- each group is defined by their GID (group id)
- users can be part of multiple groups, but each user has a **primary group** assigned to it

information about user and groups

```
# get username of current session
[user@linux ~]$ whoami
user

# get groups of current user
[user@linux ~]$ groups
user adm dialout cdrom

# get UID and all associated GIDs
[user@linux ~]$ id
uid=1000(user) gid=1000(user)
groups=1000(user),4(adm),20(dialout),24(cdrom)
```

5 Users, Groups & Permissions - Commands 2

creation / removal of users / groups (requires `root` privileges)

```
# creating user + setting password
[user@linux ~]$ useradd -m -s /bin/bash <username>
[user@linux ~]$ passwd <username>

# removing user
[user@linux ~]$ userdel -r <username>

# create group
[user@linux ~]$ groupadd <groupname>

# removing group
[user@linux ~]$ groupdel <groupname>

# add user to group
[user@linux ~]$ usermod -a -G <groupname> <username>
```

5 Users, Groups & Permissions - /etc/passwd

```
[user@linux ~]$ cat /etc/passwd
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
...
kali:x:1000:1000:,,,:/home/kali:/usr/bin/zsh
```

5 Users, Groups & Permissions - Introduction

- Linux supports multiple-users
- there needs to be rules in place in order to decide which user can access which resource
- the following two properties of a resource determine the access
  - file ownership
  - file permission

5 Users, Groups & Permissions - Basics of Permission

- whenever a file is created by some user, there are three sets of permissions for this file
  - **owner**: permissions for the owner of the file (creator of file)
  - **group**: permissions for the group of the file (primary group of creator)
  - **other**: everyone who does not fall into the first to categories

- when it comes to basic permissions, for each of these groups, we need to define the permission in respect to one of the following actions:
  - **read**
  - **write**
  - **execute**

- results in a total of **nine** (basic) permissions that need to be defined for every file

5 Users, Groups & Permissions - Permissions Visualized

- let's consider a (visualized) example of this concept

example.txt	read	write	execute
owner of file			
part of the group			
anyone else			

- there are two ways in which permissions of a file can be represented:
  - numeric representation
  - symbolic representation

5 Users, Groups & Permissions - Symbolic Representation

example.txt	read	write	execute
owner			
group			
other			

example.txt								
owner			group			other		
r	w	x	r	w	x	r	w	x
1	1	1	1	1	0	1	0	0
r	w	x	r	w	–	r	–	–

symbolic →

5 Users, Groups & Permissions - Symbolic Representation

example.txt	read	write	execute
owner			
group			
other			

example.txt								
owner			group			other		
r	w	x	r	w	x	r	w	x
1	1	1	1	1	0	1	0	0
numeric →	7		6			4		

switching users and executing with permissions of different user

```
# switching users
[user@linux ~]$ su [-] bob

# execute command with privileges of different user/group
# (default: user "root" and group "root")
[user@linux ~]$ sudo <command>
```

displaying/changing ownership/permissions of a file

```
# display permissions and ownership
[user@linux ~]$ ls -la example.txt
-rw-rw-r-- 1 user user 0 Sep 16 02:54 example.txt

# change ownership of file
[user@linux ~]$ chown <user>:<group> example.txt

# change permissions using numeric representation
[user@linux ~]$ chmod 755 example.txt
```

displaying/changing ownership/permissions of a file

```
# change permissions using symbolic representation
# who: {u, g, o, a} | +: add  | -: remove | =: set
# permissions: {r, w, x}
[user@linux ~]$ chmod [who][+-=][permissions] example.txt

# examples
[user@linux ~]$ chmod u+x example.txt
[user@linux ~]$ chmod g-w example.txt
[user@linux ~]$ chmod u=rw,g=r,o=r file.txt
```

Linux & Bash Basics

"Or: How to actually use your system"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

- every command spawns a new process
- generally, processes inherit the privileges and permissions of the creator
- processes are uniquely referenced by a 5-digit id, called the PID
- first process had PID 1, next one 2, ...

display/monitor processes

```
# show processes associated to current shell
[user@linux ~]$ ps

# show all processes (other users & without shell context)
[user@linux ~]$ ps aux

# show all processes in real time
[user@linux ~]$ top
```

6 Process Management - Foreground & Background

- processes can be executed in two forms:
  - foreground
    - receives input from and sends output to a terminal
    - occupies the terminal
  - background
    - runs independently of the terminal
    - does not occupy terminal
- processes can also be suspended, i.e, stopped

basic process management 1

```
# execute process in foreground
[user@linux ~]$ sleep 50

# execute process in background
[user@linux ~]$ sleep 51 &
[1] 66940

# suspend a process through pressing Ctrl + Z
[user@linux ~]$ sleep 52
^Z
[2]+  Stopped                 sleep 52
[user@linux ~]$ sleep 53
^Z
[3]+  Stopped                 sleep 53
[user@linux ~]$ sleep 54
^Z
[4]+  Stopped                 sleep 54
```

basic process management 2

```
# display status of all jobs (= processes associated with current shell)
[user@linux ~]$ jobs
[1]   Running                 sleep 51
[2]   Stopped                 sleep 52
[3]-  Stopped                 sleep 53
[4]+  Stopped                 sleep 54

# background latest process
[user@linux ~]$ bg

# background specific process
[user@linux ~]$ bg %<job-id>

# foreground latest process
[user@linux ~]$ fg

# foreground specific process
[user@linux ~]$ fg %<job-id>
```

6 Process Management - Killing a Process

- Linux communicates with processes by sending *signals*
- to end (*kill*) a process, there are two signals that Linux can sent
  - `SIGTERM` (Signal Terminate): asks the process to terminate, allowing it to clean up before exiting
  - `SIGKILL` (Signal Kill): forcefully kills the process immediately

killing processes

```
# kill a process via SIGTERM
[user@linux ~]$ kill <pid>

# kill a process via SIGKILL
[user@linux ~]$ kill -9 <pid>

# kill a job via SIGTERM
[user@linux ~]$ kill %<job-id>

# kill a job via SIGKILL
[user@linux ~]$ kill -9 %<job-id>
```

6 Process Management - Services

- a service is a type of long-running processes, that run in the background and waits for requests or events to handle
- also referred to as *daemons*
- started by the *init system* (initialization system), also responsible for booting the system (PID is `1`) at boot time
- common examples
  - SSH server (`sshd`)
  - web server (`apache2`, `httpd`)
  - databases (`mysql`, `mariadb`)
- `systemctl` utility used to manage them, older version rely on `service`

managing services

```
# list all services
[user@linux ~]$ systemctl list-units --type=service

# show status of a service
[user@linux ~]$ systemctl status <service>

# start/stop/restart service
[user@linux ~]$ sudo systemctl start service
[user@linux ~]$ sudo systemctl stop service
[user@linux ~]$ sudo systemctl restart service

# enable a service (i.e., should start at boot)
[user@linux ~]$ sudo systemctl enable service
```

Linux & Bash Basics

"Or: How to actually use your system"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

7 Networking Commands

- we will introduce a variety of networking utilities used for monitoring and managing all network related topics on Linux

Utility	Description
`ping`	check connectivity to host; `ICMP Echo` request
`traceroute`	trace path packets take to a network destination
`dig`	dns client, alternative: `host`
`ip`	configuring and monitoring interfaces, routing, …; deprecated version: `ifconfig`
`ss`	shows socket statistics, i.e., allows to view active ports; deprecated version: `netstat`
`iptables`	configuring firewall; frontend version: `ufw`
`tcpdump`	capture and analyze network traffic

`ping`, `traceroute` & `dig`

```
# ping a host
[user@linux ~]$ ping <host>

# ping a host n times
[user@linux ~]$ ping -c <n> <host>

# trace a route to a host
[user@linux ~]$ traceroute <host>

# dns query for a hostname
[user@linux ~]$ dig <hostname>
```

`ip` & `ss`

```
# list interfaces
[user@linux ~]$ ip address

# display routing table
[user@linux ~]$ ip route

# show neighbours (i.e., ARP table)
[user@linux ~]$ ip neigh

# show all sockets
[user@linux ~]$ ss -a

# show all listening TCP & UDP ports and which process occupies it
[user@linux ~]$ ss -tulpn
```

tcpdump

```
# capture all traffic on specific interface
[user@linux ~]$ tcpdump -i <interface>

# capture TCP|UDP|ICMP traffic on specific interface
[user@linux ~]$ tcpdump -i <interface> [tcp|udp|icmp]

# more specific capture
[user@linux ~]$ tcpdump -i eth0 src 192.168.1.1 and dst 192.168.1.2
and dst port 80
```

Linux & Bash Basics

"Or: How to actually use your system"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

- while browser is a great tool for interacting with web applications, you will also need to know how to do so from a terminal
- focus of a later session

```
# download a web resource to a specific location
[user@linux ~]$ wget http://dcelik.net/index.html -O /tmp/index.html
[user@linux ~]$ curl http://dcelik.net/index.html -o /tmp/index.html
```

- regarding communication, will introduce two ways of building communication channels with a different system

```
# unencrypted communication channel to a different system
[user@linux ~]$ nc 1.2.3.4 <port>

# ssh communication channel to a different system
[user@linux ~]$ ssh user@1.2.3.4 [-i id_rsa]
```

- ssh allows for having a shell on a different system (if configured)
- authentication occurs through username and either
  - password-based authentication
  - public key-based authentication

Linux & Bash Basics

"Or: How to actually use your system"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

9 Shortcuts - Terminal

shortcut	description
`Ctrl + L`	clear terminal screen
`Ctrl + Shift + C`	copy selection
`Ctrl + Shift + V`	paste clipboard
`Ctrl + Shift + N`	open new terminal window
`Ctrl + Shift + T`	open new terminal tab

9 Shortcuts - General Shell Shortcuts

shortcut	description
`Ctrl + C`	stop current process/command
`Ctrl + D`	log out of current shell session / exit terminal
`Ctrl + Z`	suspend current process

9 Shortcuts - Bash-Specific Shortcuts 1

shortcut	description
`Ctrl + A`	move cursor to beginning of line
`Ctrl + E`	move cursor to end of line
`Ctrl + W`	erase the word before the cursor position
`Alt + D`	erase the word after the cursor position
`Ctrl + U`	remove everything before cursor position
`Ctrl + K`	remove everything after cursor position
`Ctrl + R`	search for command in history

9 Shortcuts - Bash-Specific Shortcuts 2

shortcut	description
`Up Arrow Key`	cycle through previous commands in history
`Down Arrow Key`	cycle through next commands in history
`Tab`	auto-complete a command/filename
`Tab twice`	list all possible completions

Linux & Bash Basics

"Or: How to actually use your system"

1	The Linux Operating System
2	Linux File Structure
3	Bash Basics
4	Stream Control
5	Permissions
6	Process Management
7	Networking Commands
8	Transfer & Communication
9	Shortcuts
10	Useful Tools

Tip: click any entry to jump to that section

10 tldr

- while the `man` utility should be your go-to option for learn about a specific utility, the `tldr` utility can come in handy when wanting a snapshot of the most important functionalities

```
# installing tldr
[user@linux ~]$ sudo apt install tldr

# display short summary of a command
[user@linux ~]$ tldr <command>

# display short summary regarding a subcommand
[user@linux ~]$ tldr <command> <subcommand>
```

10 tmux - Introduction

- tmux is a terminal-multiplexer
  - multiplexers allow for multiple terminal sessions to be accessed, managed, and controlled from a single screen
  - great type of tool for productivity and overview
- alternatives: Screen and Byobu

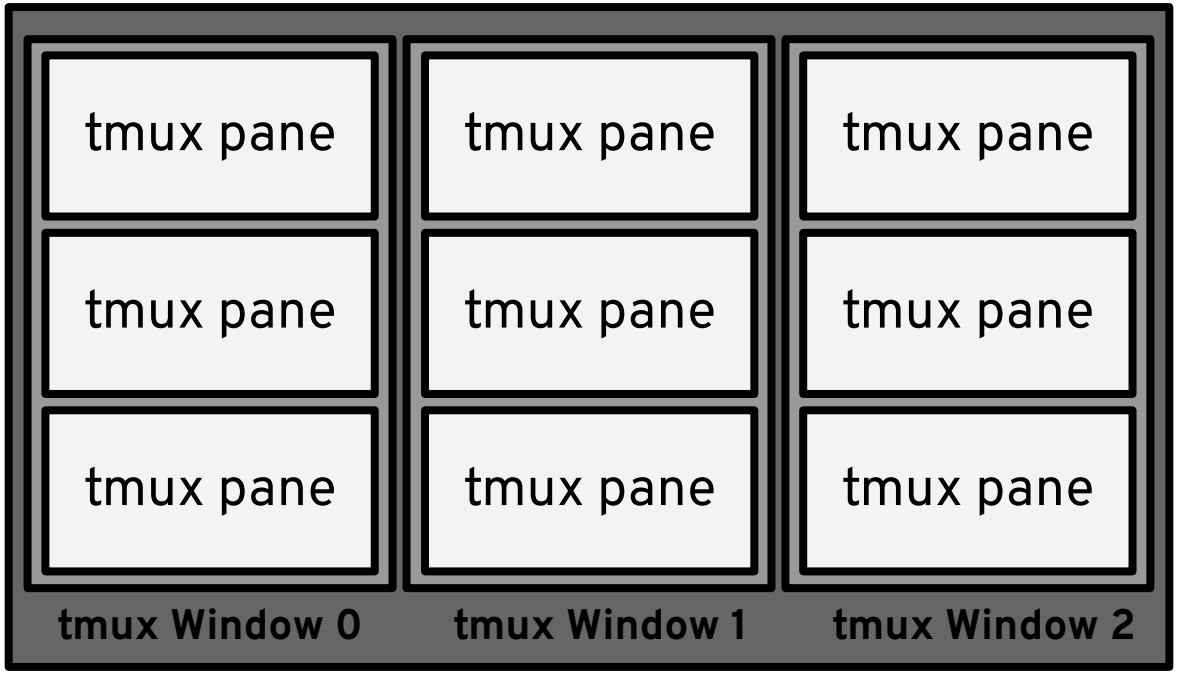
10 tmux - Setup

- start by installing it

```
# installing tmux
[user@linux ~]$ sudo apt install tmux
```

- while it is ready to use, I recommend applying some configuration change
  - a good start is the configuration file outlined by Ham Vocke in his blog
  - be sure to read what the changes do
- we will assume that this configuration file is used

10 tmux - Hierarchy



tmux

Session

10 tmux - Commands

tmux

```
# start a new tmux session
[user@linux ~]$ tmux

# start a new named tmux session
[user@linux ~]$ tmux new -s name

# list existing tmux sessions
[user@linux ~]$ tmux ls

# attach to the most recently detached session
[user@linux ~]$ tmux attach
```

10 tmux - Shortcuts 1

shortcut	description
`Ctrl + a`	prefix; default: `Ctrl + b`
`Ctrl + a c`	create new window
`Ctrl + a w`	display all open windows
`Ctrl + a [0|...|9]`	select window `0`/…/`9`
`Ctrl + a -`	split window horizontally; default: `Ctrl + a “`
`Ctrl + a |`	split window vertically; default: `Ctrl + a %`
`Ctrl + a x`	close current pane
`Ctrl + a d`	detach from current session

10 `tmux` - Shortcuts 2

shortcut	description
`Alt + arrow`	switch between panes; default: `Ctrl + b arrow`
`Ctrl + a ,`	rename window
`Ctrl + a z`	expand/shrink active pane

(Re)Sources

- https://www.linuxfoundation.org/blog/blog/classic-sysadmin-the-linux-filesystem-explained
- https://tryhackme.com/module/linux-fundamentals
- https://tmuxcheatsheet.com/